ChatGPT

RuneDecrypterPrime Codebase Forensic Audit

This audit identifies **subtle bugs, broken assumptions, and ambiguous contracts** in the RuneDecrypterPrime (RDP) codebase that could derail complex cryptanalysis solves. The focus is on scoring precision, solver logic, key operations (KeyOps) fidelity, and representation integrity (e.g. WLI semantics). Each issue is described with evidence from the code, followed by severity and recommendations. A summary of **integration tests** and **open design questions** is included at the end, along with clear **recommendations** for code and contract improvements.

Scoring Pipeline and Precision Issues

Overview: The scoring pipeline exhibits several weaknesses that can silently degrade solver performance on hard problems. These include mis-specified objective direction, enforced low-precision floats, and improper handling of optional WLI (Word Location Index) inputs. These issues risk non-deterministic or incorrect score comparisons, causing plateaued solves or misguided search.

`rune_scorer.py` – Objective Direction Misalignment (NEGLOGP vs Maximize)

RDP's scoring configuration provides an `ObjectiveFamily` (e.g. `NEGLOGP` for negative log-likelihood) and a boolean `maximize` flag. In practice, **solvers assume "higher score = better" regardless of the flag**, creating an implicit contract that all objectives must be maximized [1]. However, the NEGLOGP objective returns the *negative* average log-probability [2] – meaning a lower (more negative) raw value is better, but the solver will mistakenly treat higher (less negative) values as better. This **silent sign error** can cause solvers to optimize in the wrong direction while telemetry appears normal. The solver's accept/reject checks (e.g. using `>` for improvements) are all geared for maximize-mode [1], so a minimizing objective violates assumptions.

Evidence: In code, the `ScoringConfig.maximize` flag is defined but never enforced; solvers always sort and compare scores in descending order [1]. Meanwhile, the NEGLOGP implementation explicitly negates the log-probability before returning [2]. This mismatch means if NEGLOGP is used, the solver believes it's maximizing a positive score when it should be minimizing a cost.

Impact: This issue is *solver-blocking* if a minimizing objective (like NEGLOGP) is ever used without modification [3]. The search may diverge or converge to incorrect solutions, because it is effectively *rewarding* worse solutions.

Recommendation: Either disallow use of NEGLOGP (or any minimizing objective) with the current solvers, or update solver logic to honor the `maximize` flag. At minimum, add a **guard** to raise an error if a minimize-type objective is combined with an assume-maximize solver [4]. An integration test should construct two scores (one clearly better in a minimizing sense) and verify the solver correctly prefers the true optimum or fails loudly. (See **Proposed Tests** below.)

`rune_scorer.py` & `unified_rune_scorer.py` – Floating-Point Precision Enforcement (Float32 vs Float64)

Throughout the scoring pipeline, scores are **downcast to 32-bit floats**, undermining precision on tough differentiations. Two critical points stand out:

- **ECDF interpolation:** When scoring uses ECDF (empirical CDF) normalization, intermediate calculations cast inputs to `np.float32` and produce float32 outputs [5]. Even if a higher precision (`float64`) is requested, the interpolation forces 32-bit results.
- **Batch scoring adapter:** The `UnifiedRuneScorer` wrapper unconditionally casts batch scores to float32 [6], regardless of the backend's dtype or user preference. Additionally, the NumPy batch scorer pre-allocates an output array in float32 [5].

Evidence: The code explicitly uses `np.asarray(..., dtype=np.float32)` during ECDF score computation [5] and returns results as float32. In the unified scorer, we see `return np.asarray(self._backend.batch_score(...), dtype=np.float32)` [6], ensuring that even a float64 backend result is truncated. This means a user setting `dtype="float64"` in scoring config will **not** actually get 64-bit precision where it matters [5].

Impact: On easy puzzles, float32 may be sufficient. But for **hard multi-phase ciphers with tiny score differences**, this can be *solver-blocking*: near-tie candidates can become indistinguishable due to precision loss, causing plateaus or incorrect beam ordering [7] [8]. Non-deterministic tie-breaking can also occur if scores quantize to identical float32 values when they differed in float64.

Recommendation: Honor the `dtype` setting end-to-end, or at least document that all internal scores are float32. Ideally, the unified scorer should preserve backend precision or perform comparisons in higher precision before casting for storage. A dedicated test should feed two nearly-equal scoring candidates (with a very small known difference) and assert that a float64 pipeline can discern the better candidate while float32 might not [9].

`rune_scorer.py` – WLI Model Activation Without WLI Data (Missing-Data Handling)

The scoring pipeline includes models that use WLI (word-location index) features (e.g. word position or word-length signals). **Currently, these WLI-dependent models are activated based on config flags rather than actual WLI availability**, leading to inconsistent behavior if WLI data is missing. Specifically, if `use_word_breaks=True` (meaning WLI features desired) but no WLI array is provided (or it's empty), the scorer will:

- Include WLI-based submodels in the weighted ensemble (affecting normalization of weights) [10].
- During scoring, skip applying those models (since `wli` is `None`), effectively treating their contribution as constant or zero while still having reserved weight [11].
- In extreme cases (if **only** WLI-driven models are enabled and WLI is missing), the score can collapse to a constant baseline with **no error raised**, misleading the solver with flat gradients [12] [13].

Evidence: In the NumPy scorer path, `_active_models()` builds the model list purely from config (e.g., include WLI model if `use_word_breaks=True`) [10]. Later, when computing scores, the code checks if `wli` data for a given model is present; if not, it *skips contribution but keeps the normalized weight* [11]. This

means the sum of weights still includes a model that contributed nothing, altering the scale of other model contributions [14]. The Torch scorer path has a similar logic (setting `wli_t = None` and continuing) [15]. There is **no runtime error or warning** in these cases.

Impact: This is *solver-degrading* in normal mixed-model cases (scores shift because weights are mis-normalized) and *solver-blocking* if the user configures a WLI-only scoring objective without providing WLI [16] [17]. In the latter scenario, the solver would see almost no variation in scores (all candidates get the same flat score), preventing any meaningful convergence.

Recommendation: The scorer should detect the mismatch and either (1) drop WLI models entirely from both weight normalization and scoring if no WLI is present, or (2) raise a clear error if `use_word_breaks=True` but WLI data is missing. The second approach is safer to catch misconfigurations. An integration test should attempt a run with `include_char=False, use_word_breaks=True` and no WLI, and expect a failure or identical outcome to a run where WLI is explicitly disabled (no silent difference in scaling) [18].

`language_model_prime.py` – Language Model Smoothing Cache Contamination

RDP's language model scorer caches loaded n-gram tables by file path. A hidden issue is that **the cache does not account for smoothing or OOV (out-of-vocabulary) settings**, and the native code *modifies the cached data in place* when applying smoothing. This means if two scorers load the same model file with different smoothing parameters, the first scorer's settings can **bleed into the second's results** without any indication.

Evidence: The `_load_bin` function uses a module-level cache `_load_bin_cache` keyed only by the file path [19]. If a path is in the cache, it returns previously loaded arrays *by reference*. The native `FastTransitionModel` in `fastlm.cpp` takes smoothing parameters (like Laplace alpha or Good-Turing flags) and **overwrites the `logp` probability array in place with smoothed values** [20]. Thus, a second scorer that retrieves the same cached `logp` array will get it *already smoothed* with potentially different parameters than it asked for. There is no versioning by smoothing/OOV policy.

Impact: This cross-contamination is *solver-degrading* and **audit-hostile** [21] [22]. For example, a scorer configured with no smoothing might unexpectedly receive Good-Turing smoothed probabilities if another scorer earlier in the process used that model with smoothing. This can shift score values mid-run or between runs, confusing tuning and making results irreproducible across different scoring configurations.

Recommendation: Isolate language model assets per config. The cache key should include smoothing parameters (e.g. `(path, smooth_mode, oov_mode, alpha)` as suggested) [23], or the cached arrays should be immutable originals that each scorer copies and smooths on its own. A test for this could instantiate two scorers (one with smoothing off, one on) on the same model and verify that the first scorer's results remain consistent before and after using the second scorer (no drift in scores) [24].

Solver Logic and Key Mutation Issues

Overview: The solver engines (Simulated Annealing, Beam Search, Genetic Algorithm, etc.) generally assume properly-behaved KeyOps and scoring outputs. We found discrepancies in random seed

determinism, silent handling of invalid seeds, and mismatches between documented and actual solver parameters. These can make solver behavior inconsistent or unpredictable, especially for multi-phase or seeded solves.

`pipeline.py` & `solver_engine.py` – Inconsistent Determinism Defaults

RDP's main pipeline enforces a default RNG seed of 0 (for reproducibility) when none is provided, whereas the legacy `SolverEngine` usage will use a fresh entropy-based RNG if no seed is injected. In other words, **the same solve request can be deterministic or not depending on which API entry point is used**.

Evidence: In `RunAPI.run()`, the code normalizes the seed: if no seed is given, it explicitly sets `effective_seed = 0` [25]. This flows into the engine, which then uses `np.random.default_rng(0)` – deterministic sequence each run [25]. Conversely, if one bypasses the high-level pipeline and uses the lower-level `core.solver_engine`, that code calls `np.random.default_rng()` with no fixed seed [26], yielding different sequences each time. The audit confirms this discrepancy [27] [28].

Impact: This is *solver-degrading* for debugging and reproducibility [29] [30]. A user might assume "same inputs, same outputs," but if they inadvertently use a different code path (e.g. a legacy interface or custom loop), results can appear flaky. Hard cryptanalytic solves may fail intermittently due to uncontrolled randomness, undermining trust in the tool.

Recommendation: Unify determinism policy: ideally, default to a fixed seed (0 or a documented constant) in *all* entry points unless the user specifies otherwise. At minimum, warn when a non-deterministic path is used. A test should run the same solve twice via the legacy path with no seed and expect identical results or a clear warning [31].

`solver_base.py` – Silent Seed Key Replacement on Normalization Failure

The solver initialization accepts user-provided initial key guesses ("seeds"). If a seed is invalid (e.g., wrong length or out-of-domain), the code **silently replaces it with a random key** without notifying the user unless even the random replacement fails. This behavior hides potentially critical input errors and changes the solver's starting conditions unexpectedly.

Evidence: In `SolverBase.__init__`, the provided `initial_keys` are normalized via `self.keyops.normalize()`. If normalization fails for a seed row (throws exception or returns None), the code catches it, logs a debug message, and generates a new random key via `self.keyops.random(...)` to use instead [32]. Only if *that* also fails does it increment a "dropped seed" counter [33] [34]. Furthermore, when using composite ciphers (e.g., with interruptors), a valid seed must match the full composite key length (core key plus interruptor part); the code does not explicitly check this upfront, so a too-short seed will be treated as invalid and randomly expanded [35]. Telemetry still counts these as "provided" seeds unless dropped, masking the event [36].

Impact: This is *solver-degrading*: a user might supply a specific starting key (perhaps from a prior run or an intuition), but the solver could ignore it and start elsewhere without clear indication. Particularly for composite ciphers, a user might not realize they needed to provide a longer tuple key. Debugging performance or behavior issues becomes hard when the initial conditions are not what the user intended.

Recommendation: Fail fast and loudly on invalid seed keys. If a seed cannot be normalized (especially length mismatches in composite keys), the system should raise an error or at least record a telemetry flag (e.g., `seed_normalize_replaced`) that is clearly visible [37]. Proposed tests: (a) feed an incorrect-length seed for a known composite cipher and assert that the run either errors out or flags the replacement in results; (b) feed valid seeds and verify the solver's internal starting pool matches exactly those seeds (no unintended alteration).

`solvers/beam.py` – Beam Expansion Parameter Mismatch (API vs Solver Implementation)

The Beam Search solver has tunable parameters for how it expands the beam each iteration (e.g., how many parent states to carry forward, whether to sample or take top scores). We found a **naming and contract mismatch** between what the public API expects and what the solver actually uses:

- The API (via `SolverSpec` and `_resolve`) defines beam parameters like `expand_mode`, `top_parents_factor`, and `sample_per_parent` as the canonical keys for beam configuration [38].
- The Beam solver code, however, looks for parameters under an `expand.*` namespace (e.g., `expand.parent_mode`, `expand.parents_frac`) and also always uses *deterministic top-k parent selection* internally [39] [40].
- The solver's `_normalize_expand_params()` reads the API-style keys (`expand_mode`, etc.) and translates them, but the telemetry and some parts of the code refer to the `expand.parent_*` variants [39]. For instance, you cannot directly set `expand.parents_frac` via the public API; and conversely setting `top_parents_factor` may be accepted but not fully utilized if the solver logic expects a different usage.

Evidence: The code shows that `_CANON_OPTS["beam"]` includes `expand_mode`, `top_parents_factor`, `sample_per_parent` (and *not* the `expand.parent_mode` keys) [38]. Inside `BeamSolver.solve()`, it calls `get_param("expand.parent_mode", ...)` etc., but these would only be present if `_normalize_expand_params` copied values over. In `_normalize_expand_params`, it indeed reads the canonical ones and populates internal settings. However, one key issue is that regardless of `parent_mode` ("top" vs "stochastic"), the actual expansion implementation selects parents by `np.argsort(scores)` (highest scores) every time [40] – meaning **any stochastic or fractional parent selection mode is effectively ignored**.

Impact: This misalignment is *solver-degrading*, especially for tuning. Users might try to adjust beam parameters (e.g., to add randomness in parent selection or limit beam width fractionally) and see no effect, or their config may be rejected as unknown. In complex solves, this leads to trial-and-error "folklore" tuning because documented knobs don't do what they say.

Recommendation: Clarify and unify the parameter interface. Choose one set of names (preferably the user-facing ones) as the source of truth, and ensure the solver implements all modes (or explicitly reject unsupported modes). At the very least, update telemetry to reflect the actual controlling parameters. A test can attempt to set a beam parameter in the "wrong" namespace (e.g., pass `expand.parents_frac` via API) and expect either a clear error or a confirmed effect. Another test: toggle `expand_mode` between deterministic and stochastic and verify that parent selection in the solver output actually changes (which would require implementing the stochastic behavior or else flagging that it's not supported) [41].

`keyops/composite.py` – Composite Key Length and Variable-Interruptor Handling

Composite ciphers (those that combine a primary key with interruptor positions) present tricky invariants that RDP currently does not enforce, leading to potential runtime crashes or solver dead-ends:

- **Seed length vs composite length:** For composite ciphers, the full key is typically a concatenation of core cipher key and interruptor indicators. The expected length is `core_K + interrupt_K`. However, as noted above, if seeds are provided at the core length only, the system will accept them and pad randomly rather than erroring [35]. This is partly a KeyOps contract issue: `CompositeKeyOps.normalize()` checks that provided keys match the full length and raises a ValueError if not [35], but this may be caught and treated as "need random replacement" by the solver instead of stopping.

- **Initial permutation vs interruptor removal:** If an initial text permutation is configured and interruptors are in use, there's a conflict. The API currently requires the permutation length to equal the full ciphertext length on input [42], but at runtime the cipher first removes interruptors (dropping characters) then applies the permutation, which must match the *reduced* length [43]. A permutation provided for the full length will pass initial validation (because it matches ciphertext length) but then cause a length mismatch error when applied after removal [44] [43]. Conversely, if the user tries to provide a permutation of the core length (to match after removal), the initial validation will reject it as wrong length [45] [46]. In short, there is **no way to supply a valid permutation when using interruptors**, due to a structural contract bug.

- **Variable interruptor count:** Some composite KeyOps allow the number of interruptors to vary per key (e.g., between `min_count` and `max_count`). This means different candidate keys will produce different core text lengths. A single fixed permutation cannot consistently apply to all, since the core length isn't constant [47] [48]. RDP does not currently detect or prevent this scenario, so a solve that mutates interruptor count while holding an initial permutation could crash mid-search with a "text_perm must match text length" error when a key's interruptor count deviates [49] [50].

Evidence: The composite KeyOps code explicitly randomizes interruptor count within a range [51], and uses a special sentinel value to mark interruptor positions [52]. The permutation application code throws an error if lengths don't match [53]. The pipeline docs and code clearly expect permutations to be defined after interruptor removal (core space) [54] [55], but the implementation doesn't allow specifying it that way. Also, `ProblemSpec` validation demands permutation length == original text length [45]. For interruptor pool values, the config only checks that indices are non-negative and unique [56], and actual range validation (`idx < length`) happens only at decrypt time [57], i.e. after the solver may have been running for a while.

Impact: These issues are *solver-blocking* whenever a user tries to combine text permutations with interruptors [58] [59], or uses variable-count interruptors with any fixed permutation [48] [60]. The solve will either refuse to start (if checks catch it) or, worse, crash at runtime when a length mismatch arises mid-optimization. The interruptor pool range issue means a misconfigured pool (e.g., containing an index equal to ciphertext length) will not be caught until a specific key triggers an out-of-range removal, causing a late failure [61] [62].

Recommendation: Strengthen contract checks and clarify usage: - Enforce that if `initial_text_permutation_indices` is used in a cipher with interruptors, the interruptor count must

be fixed and the permutation length must match the core length (and make the API accept core-length permutations). If variable counts are desired, then initial permutation should be disallowed (or the permutation must be applied before interruptor removal, which contradicts current design). - Validate interruptor pools against ciphertext length at config time, not just at runtime [63] [57] . - If the user supplies a permutation for a cipher with interruptors, either automatically adjust it to core length if possible or throw an error upfront explaining the issue (to avoid the cryptic late `text_perm` error).

Tests should cover: (a) using an initial permutation with interruptors and expecting a clear error that references the incompatibility (not a generic length mismatch at runtime) [64] ; (b) varying interruptor count with a fixed permutation and ensuring the system halts with an explanation [65] ; (c) an interruptor pool containing an out-of-range index should be caught before the solver starts (e.g., in `InterruptorConfig` or ProblemSpec materialization) [66] .

Naming and Representation Bugs

Overview: Several variables and structures in RDP carry ambiguous or misleading names, leading to subtle bugs in how data is interpreted. Chief among these is the **WLI (Word Location Index)** representation, which has diverging definitions across the code. Additionally, telemetry and API naming inconsistencies (e.g., beam parameters noted above) can hide contract violations. We highlight the major naming/representation mismatches below.

`api/normalize.py` & Tests – WLI Semantic Mismatch (Start/End vs Pos/Len)

The **Word Location Index (WLI)** is intended to carry word-position information for each character (rune) of the ciphertext. There is a fundamental ambiguity in how WLI is represented: - The **API documentation and normalization code** treat WLI as a pair of `[start_index_in_text, word_length]` (effectively start offset and length of the word) for each character [67] . For example, in a two-letter word, both letters would get WLI `[0, 2]` according to `wli_from_text` and the docs. - The **scoring algorithms (language model, Hamming)** expect WLI as `[position_in_word, word_length]` [68] . In this scheme, the first letter of a two-letter word is `[0, 2]` and the second letter is `[1, 2]`, resetting position for each word.

Currently, `wli_from_text()` produces the **start-offset form**, giving each rune of a word the same pair [69] , and a pipeline helper then tries to convert these to spans or something, but does so incorrectly for multi-word inputs (next issue). Tests even assert the start-offset behavior as correct (which is actually the wrong contract) [69] . Meanwhile, the `Runeglish.encode_english_to_runes` reference implementation (and the underlying C++ scorer) use the pos/len form [68] .

Evidence: A test in `tests/api/test_normalize.py` explicitly checks that for the word "AB", `wli_from_text` returns `[[0,2],[0,2]]` (both A and B tagged as start=0,len=2) [69] . However, the `Runeglish.encode_english_to_runes` function populates WLI as `[0,2]` for A and `[1,2]` for B [70] . The Hamming scorer (C++ `Hamming.h`) checks for `wli[0] == 0` to detect word breaks [71] , implying the expectation that only the first character of each word has pos=0. If the API passes `[0,2],[0,2]`, the second character incorrectly looks like a word start to the scorer, potentially messing up segmentation logic.

Impact: This confusion is *solver-blocking* when WLI-based scoring is used, because it can flatten score differences or compute wrong penalties if word boundaries are mis-marked. It's also *deceptive in tests/*

documentation: the test suite currently blesses a behavior that violates the actual scorer contract [72] [73], meaning a regression could pass tests but fail in reality.

Recommendation: Decide on a single WLI interpretation (most likely **pos-in-word, len** to match the scoring code and original design) and enforce it everywhere: - Change `wli_from_text()` (and `make_single_word_wli`) to produce pos/len pairs (e.g., for "AB" -> `[[0,2],[1,2]]` not `[[0,2],[0,2]]`). - Update tests and docs to reflect this, or if start/len must be kept for some legacy reason, then convert to pos/len at the point of scoring **with an explicit, tested transformation**. - Add runtime validation: any WLI passed into scoring should be checked that `0 <= pos < len` and that each new word starts with pos=0. In fact, a check `if any(pair[1] < pair[0] for pair in wli): ...` is already used to trigger a conversion, but the logic is flawed for multi-word (see below).

A test should explicitly assert that for a multi-word input, WLI pairs reset position at each word (multiple `[0, …]` entries appear) and no position exceeds its length. Another test should feed a known WLI through the pipeline end-to-end and compare with the `Runeglish.encode_english_to_runes` result for the same text [74].

`api/pipeline_helpers.py` – WLI Span Conversion Glitch on Multi-Word Input

To reconcile the above WLI inconsistency, the pipeline calls `coerce_wli_for_config()` on any WLI list. This function checks if any WLI pair looks like `(start, length)` with `start >= length` (as a hint it might be a span) and if so, converts it to a span representation (i.e., calculates `end = start + length - 1`). This logic is meant for cases where WLI might be given as (start,end) pairs. Unfortunately, on a multi-word string input, the **start-offset form triggers this span conversion erroneously**:

- For each word after the first, the `word_start_offset` grows larger than the word's length (e.g., in text "AB CD", the "CD" word might produce WLI `[3,2]` for both letters, since that word starts at index 3 with length 2). The check `pair[1] < pair[0]` sees `2 < 3` and decides conversion is needed [75].
- It then interprets those as spans: `start=3, span=2` becomes an end index of `3 + (2-1) = 4`. So the two letters get WLI `[3,4]` instead of `[0,2]` and `[1,2]`.
- These `[3,4]` values then flow into scoring, where the scorer treats the first value as pos-in-word (3) and second as length (4). Since pos should be < length, the system masks them mod 64 (both 3 and 4 are <=63 so it might not notice immediately, but they are wrong relative to their own word).

Essentially, *every multi-word string input ends up with WLI lengths that equal absolute end indices rather than word lengths*. The scoring code will silently mask these down to 6-bit, which can cause collisions and meaningless WLI features [76] [77].

Evidence: The audit describes this in Finding 12: `wli_from_text()` emits `[word_start_offset, word_len]` for each rune [75]. Because for the second word `word_start_offset > word_len`, `coerce_wli_for_config` sets `needs_span_conversion=True` and computes an `end` index [78]. As a result, what should have been e.g. `[0,2]` and `[1,2]` for the word "CD" could turn into `[3,4]` for both letters. The scorer later applies `(pos & 0x3F)` and `(len & 0x3F)` masks to each WLI [79]; though 3 and 4 are within 6 bits, the semantic is wrong. The intended invariant `0 <= pos < len <= 63` is

broken at runtime (3 !< 4? Actually 3 < 4 is okay, but it's not pos-in-word; if a longer offset happened, e.g. 10 vs 2, it would wrap mod 64 and truly corrupt data). No explicit check catches this in the hot path.

Impact: This is a *solver-blocking bug for any multi-word input using the default string normalization*. It would flatten WLI-based score distinctions and could cause hash collisions in any hashing of WLI. Telemetry might not catch it because the values still look like numbers (just wrong semantics).

Recommendation: Remove or heavily modify the span conversion logic. If WLI is meant to always be pos/len, there should be no need to ever convert a pair by subtracting 1 or treating it as (start,end). The code at `pipeline_helpers.py:238` should likely be deleted or replaced with an assertion that no `pair[1] < pair[0]` occurs [78]. The correct fix ties into deciding WLI semantics globally (as above).

Add a test: Provide a multi-word plaintext via the API (with `use_word_breaks=True`) and verify that **every** WLI pair in the final ProblemSpec satisfies `0 <= pos < len <= 63` [74]. This will catch the explosion of WLI lengths described.

`telemetry/pipeline.py` – Permutation Length Reported in Wrong Terms

The telemetry pipeline, when logging the run configuration, includes a summary of any initial text permutation. It always reports the permutation length as the full ciphertext length, even if interruptors are in play [55]. But as discussed, the actual applicable permutation length should be the core length (after interruptor removal) for the runtime to accept it [80]. This mismatch can mislead users reviewing logs or debugging contract issues.

Evidence: Telemetry calls `_perm_summary(text_permutation, int(ciphertext_len))` where `ciphertext_len` is the full original length [55]. Meanwhile, the cipher pipeline documentation states permutations indices are after removal (core space) [80]. If a permutation is provided and interruptors are present, the telemetry might show a permutation of length N (full length) even though the runtime expects length M (core length). Or if the user tried to supply core-length, it would have been rejected but telemetry wouldn't clarify that discrepancy.

Impact: This issue is *solver-degrading* in the sense of debugging and development. Telemetry that misreports lengths can make it appear that everything is consistent even when the underlying run is headed for a failure or not doing what is intended [81] [82]. It complicates verifying that the system enforces the correct permutation contract.

Recommendation: Align telemetry with runtime semantics. Telemetry should compute the expected core length if interruptors are enabled and report permutation length in that context, or explicitly note "(full ciphertext length)" vs "(core length)" where applicable. This clarity will make debugging easier when a user mis-specifies a permutation. A test for this could set up a scenario with interruptors and a permutation and inspect the telemetry output to ensure it references the correct length space (and ideally, that the system refuses inconsistent configurations early).

`api/run.py` – English Plaintext Input Ignores Specified Encoding Direction

RDP supports both left-to-right (LTR) and right-to-left (RTL) encoding directions for languages (affecting how text is transliterated to numeric indices). However, if the user provides a plaintext string (English) to `RunAPI.run` and specifies `encoding_dir="rtl"`, the code currently **ignores the direction when converting the text to runes**, yet still applies it in scoring config. This yields a mismatch: the text is encoded as LTR by default, but the scorer might use RTL language models or statistics.

Evidence: In `RunAPI.run`, the `encoding_dir` is normalized and stored, but the call to `normalize_ciphertext(text, ...)` does not pass this direction information [83]. Inside `normalize_ciphertext`, it uses `to_indices(text)` which for English string input will break into words and use `Runeglish.translate_to_gematria` without any awareness of direction [83]. Meanwhile, later in `run()`, the `scoring_cfg.encoding_dir` is set to the user-provided direction and eventually affects scorer asset loading (choosing RTL vs LTR language model data, etc.) [83]. Thus, an English string "TEST" with `encoding_dir="rtl"` will produce indices as if LTR (T,E,S,T in normal order), but the scorer might, for example, pick an Arabic or Hebrew char frequency model expecting RTL ordering, or apply bigram models reversed.

Impact: This is *solver-degrading or even blocking* for direction-sensitive problems [84]. If the user intends an RTL cipher (e.g., a substitution cipher but text read right-to-left), the scoring will not match the encoded text, leading to nonsense evaluations. It might not be obvious to the user that the text was encoded incorrectly, since no error is thrown – it's just a silent contract violation between encoding and scoring.

Recommendation: Ensure that plaintext normalization respects `encoding_dir`. The function `to_indices` (or its caller) should take `encoding_dir` as a parameter and use the proper transliteration or reversing of order if needed. Alternatively, disallow using `encoding_dir` with raw English text input and require the user to pre-encode through `Runeglish.encode_english_to_runes` if they want non-default direction. Tests should compare the API string handling with the direct `Runeglish.encode_english_to_runes(text, direction=...)` to ensure parity. For example, `"TEST"` with `encoding_dir="rtl"` should come out as the same rune sequence as if the string were reversed before encoding or however the RTL is defined, or the system should clearly error if this is not supported.

Other Naming/Representation Concerns

- **Beam `expand` parameters:** (Already discussed above in solver section) The inconsistent naming (`expand_mode` vs `expand.parent_mode`) is a representation issue where telemetry and API surface different terms for the same concept. This should be unified to avoid user confusion.
- `user_map3` **key representation:** (Discussed below in KeyOps) The `user_map3` cipher is essentially an affine cipher with two components (k1, k2), but the API collapses it to a single number domain. The naming in config hints (using `A` for modulus) is misleading because the actual key space is `A×A`. This is more of a design flaw in representation: either treat it as two separate keys or explicitly as one number in [0, A*A). Currently, the code uses one number and then mod-reduces it, effectively ignoring the second component – a silent assumption that can block solves.

KeyOps and Cipher Fidelity Issues

Overview: KeyOps are responsible for generating, mutating, and applying keys under various cipher definitions. For cryptanalytic fidelity, they must preserve invariant structures (e.g., permutation keys remain bijective, modular keys wrap correctly, etc.). Generally, RDP's KeyOps implementations do follow expected patterns (e.g., substitution key mutation swaps two elements, preserving bijectivity). The problems identified lie in composite key handling and certain generic cipher implementations:

`generic_map_cipher.py` & `core/runtime.py` – Collapsed Key Space for `user_map3` (Affine Cipher)

The `user_map3` cipher (an affine-like cipher with presumably two key components) is supposed to have a key domain of size `A*A` (if A is the alphabet size, there are two independent components, often denoted k1 and k2 for affine). RDP's implementation, however, **collapses this to an effectively one-dimensional space of size A** by applying modulo arithmetic prematurely:

- The cipher internally computes `A_key = self.A * self.A` for the key space and expects a single integer key `kv` in `0..A*A-1`, which it then splits into k1 = kv // A, k2 = kv % A [85].
- But when decrypting, it immediately takes the provided key value mod `self.A` [86]. This means only k2 (equivalent to `kv % A`) actually affects the output; k1 is effectively always zeroed out by the mod.
- Moreover, the KeyOps hints system and API both treat the key length as 1 (since `resolve_key_length` returns 1 for any tuple spec) [87] and use the cipher's `A` as the modulo in mutation hints [88]. The solver, thinking the space is size A, will only search 0..A-1.

Evidence: The code clearly shows `key_arr = key_arr % self.A` in the cipher's decrypt routine [86], meaning any key beyond A wraps around. The GenericMapCipher constructor sets up `A_key = self.A * self.A` if a second dimension is expected, but then doesn't utilize it fully [85]. In `runtime.py` where key hints are gathered for KeyOps, it uses the cipher's `A` to inform the mutation space, not `A*A` [89]. Also, the API's `resolve_key_length` simply returns 1 for tuple specs, preventing the solver from treating the key as having two independent parts [87].

Impact: This is *solver-blocking* for any cipher meant to have two degrees of freedom like `user_map3` (e.g., Affine cipher `ax + b` mod 29). The solver will be effectively stuck exploring only variations in `b` (the second part), never changing `a` (the first part), because any attempt to change `a` is lost to the modulus. The cipher's own tables might be built for a full A*A space (and indeed the encryption/decryption logic uses both k1 and k2), so if k1 != 0 is required for the correct key, the solver will not find it [90] [91]. Telemetry might not flag anything unusual; the solver just fails to find a solution.

Recommendation: Support the full key domain: - Option 1: Represent the key as two independent components (length 2 array) and adjust KeyOps accordingly (so that `keyops.random` generates [k1, k2], mutate tweaks one or both, etc.). This way no mod reduction is needed; both parts are optimized. - Option 2: Keep encoding as single number 0..A*A-1 but* do not mod-reduce before decrypt*. Instead, compute k1 and k2 from the full number so that k1 can be >0. The KeyOps hints should use `mod = A*A` for this cipher's family so the solver explores the full range [89]. - The API should not collapse tuple key lengths to 1

without ensuring the cipher logic accounts for it. If tuple spec is given, perhaps the cipher should treat it explicitly.

Add a test for reachability: define a simple user_map3 cipher where the plaintext changes if k1 changes (with k2 fixed). Feed a key with k1>0 via a known solution or brute force, and ensure the solver's search space or normalization can produce that key (i.e., the solver isn't stuck with only k1=0). If not, the test should fail, indicating the need for the above fix [92].

KeyOps Validations and Assumptions

- **Permutation KeyOps (Substitution cipher):** The `PermutationKeyOps` class mutates keys by swapping two positions, correctly preserving the bijection (no duplicates) – this is good. We did not find a direct bug here; however, it's crucial that any new mutation does not violate permutation integrity. Current code appears consistent with cryptanalytic assumptions (swaps for substitution, flips for bits, etc.). No action needed aside from ensuring any new key family follows suit.
- **Range and content validation:** For interruptor-related KeyOps, we noted that invalid pool values (negative or duplicates) are caught, but values beyond text length are not caught until use [63]. This should be tightened in config as recommended. Composite key normalization ensures correct length but the solver should surface the error rather than substituting a random key (addressed above).
- **Naming of KeyOps hints:** Ensure that KeyOps hint parameters (like "A") truly reflect the key domain. In the user_map3 case, `hints["A"]` was misleading. Similarly, if a cipher's key space isn't 0..N-1, hints should clarify (e.g., for a tuple key, hints might list separate domains).

Hidden Hardcoded Limits and Assumptions

Throughout the issues above, several **hardcoded assumptions** emerged that may limit generality or introduce silent errors if not satisfied:

- **WLI 6-bit limit:** The scoring pipeline only allocates 6 bits for WLI position and length (masking with `0x3F`) [93]. This means it assumes no word will exceed length 63 and positions < 63. This is normally fine (words of length >63 are rare), but if it ever happens or if a bug creates a larger number, the values wrap and collide silently. A runtime check on WLI values (as the strict validator in `language_model_prime.py` suggests [94]) should be always on to catch this.
- **Default alphabet and modulus assumptions:** Many ciphers assume a certain alphabet size or range (like 29 for Runic, 26 for English). For example, the affine mod-29 cipher assumes working mod 29. These are generally configured via cipher specs. No explicit issue found beyond the `user_map3` collapse, but it's worth ensuring that if a user changes an alphabet length, all related components (scoring, keyops hints, etc.) adjust accordingly.
- **Default "period" lengths:** Some periodic ciphers might assume a default period if not provided. Ensure that a missing config yields a clear error or default that is documented (not a silent fallback that could be wrong for certain ciphers).
- **Fixed random seed = 0 vs true randomness:** The choice to default to seed 0 is effectively a hardcoded behavior favoring determinism. It's intentional, but users need to know this (and the discrepancy with legacy path was noted).
- **Score Plateau Delta:** Solvers use a `plateau_min_delta` (small float) to decide if an improvement is significant [95]. Because floats are float32 internally, if `plateau_min_delta` is extremely small, it might be treated as zero. This coupling of an implicit tolerance to float precision is a design

assumption. If truly needed for double precision, this should scale or the comparison be done in higher precision.

These assumptions don't all require changes but should be **documented and monitored by tests**. For example, tests could verify that `language_model_prime._validate` catches any WLI outside 0..63 (perhaps by injecting a fake large WLI and expecting a failure), or that using a non-standard alphabet size in a known cipher produces correct behavior.

Proposed Integration Tests (Robustness Checks)

To catch the above issues and prevent regressions, we propose the following **integration tests** (with suggested file locations and scenarios):

1. **WLI Invariant Canary** – *File:* `tests/api/test_wli_invariants.py` – **Scenario:** Call the high-level API with a multi-word plaintext (both as runes and as English string) while `use_word_breaks=True`. Assert that the resulting `ProblemSpec.wli_data` contains only pairs that satisfy the contract `0 <= pos < len <= 63`. Specifically, check that multiple words yield multiple `pos=0` entries (each word start) and that no pos value equals 0 for two consecutive characters unless the second is a new word [74]. This catches WLI format drift and span conversion bugs.

2. **WLI vs Runeglish Parity** – *File:* `tests/api/test_wli_parity.py` – **Scenario:** For random English texts (single and multi-word), compare the WLI output of the API's internal `normalize_ciphertext` (with no pre-provided WLI) against `Runeglish.encode_english_to_runes(text)` which returns runes and WLI. They should match exactly for both rune sequence and WLI pairs [96]. Failing this indicates an API encoding or WLI assembly bug (e.g., the span conversion or direction issues).

3. **WLI Permutation Alignment** – *File:* `tests/core/test_permutation_wli.py` – **Scenario:** Configure a cipher run with `use_word_breaks=True` and an `initial_text_permutation_indices` (e.g., shuffle the text). Ensure that scoring a given key with and without applying the same permutation to WLI yields identical results. In practice, if WLI isn't permuted in the code, the test should detect a difference, flagging the misalignment [97]. This test will catch the omission of WLI permutation when text is permuted.

4. **Objective Direction Canary** – *File:* `tests/solver/test_objective_direction.py` – **Scenario:** Use a simple cipher (or a dummy scoring function) where we know the ground-truth "better" and "worse" scores. For a solver (e.g. Beam or SA), feed two candidates where one should be preferred in a minimize context (e.g., NEGLOGP) but has a numerically lower score. Check that the solver picks the correct one if objective is configured properly, or that it errors out if an incompatible combination is set [4]. This ensures no silent inversion of objectives.

5. **Score Precision Near-Tie** – *File:* `tests/scoring/test_score_precision.py` – **Scenario:** Construct a scenario with two ciphertexts or keys that yield extremely close scores (differences on the order of 1e-7). Run scoring in batch (through `RuneScorer.batch_score`) with `dtype=np.float64` vs default. Verify that in float64 mode the ordering of scores is correct and

that float32 (if forced internally) does not invert the order [9]. If float32 must be used, this test at least documents the potential tie issue and can alert if future changes inadvertently worsen precision.

6. **WLI Missing-Data Guard** – *File:* `tests/scoring/test_wli_missing.py` – **Scenario:** Configure a scoring config with `use_word_breaks=True` but do not supply any WLI (simulate what happens if a user passes a string without WLI, or manually set `wli=None`). If `include_char=False` (no character model, only WLI model), assert that scoring either raises an error or returns identical scores for all inputs (and that such a scenario is documented) [18]. If `include_char=True` (mixed models), compare the scores in two cases: one with WLI provided and one without (but weights normalized as if WLI were there). They should either be effectively the same (if code handles it) or the difference should trigger a failure. This catches the silent WLI weighting bug.

7. **Legacy vs Main Determinism** – *File:* `tests/solver/test_determinism.py` – **Scenario:** Run a small cipher solve through the main API twice with no seed (should yield same result both times, since it defaults to 0). Then run the same solve through the lower-level solver engine interface twice (without explicitly seeding) – expect either a deterministic result or a clear warning that it's non-deterministic. If currently non-deterministic, this test should mark that difference, prompting a fix or at least documentation [98].

8. **Seed Replacement Detection** – *File:* `tests/solver/test_seed_validation.py` – **Scenario:** For a composite cipher (one with interruptors or multi-part key), supply an initial seed key that is incorrectly formatted (e.g., only core part without interruptor part). Expect the run to fail validation or at least produce a telemetry output indicating the seed was replaced. Also test a valid seed to ensure it remains unchanged through normalization (no randomization). This catches silent seed corrections [37].

9. **Interruptor and Permutation Contract** – *File:* `tests/core/test_interruptor_permutation.py` – **Scenario:** (a) Configure a cipher with a known interruptor (e.g., a simple fixed-position mask) and set `interruptors_exact` to remove one character. Provide an initial permutation of full length. Expect a ValueError upfront stating the incompatibility (not a generic one from deep inside) [64]. (b) Conversely, attempt to provide a permutation of core-length (ciphertext length minus one) when an interruptor is present – expect either acceptance (if design changes) or an error that explicitly mentions core length. This ensures the system doesn't just blindly enforce full length when it's not applicable.

10. **Variable Interruptor Count Guard** – *File:* `tests/core/test_variable_interruptors.py` – **Scenario:** Use an `InterruptorConfig` with `min_count < max_count` (variable count) and set an initial permutation. Run a small search (even one step) and ensure that either the solver refuses to start or as soon as two keys with different counts are generated, a controlled exception is raised (with a meaningful message) [65]. The test should not allow a random crash – it should capture the intended guardrail.

11. **Telemetry Permutation Reporting** – *File:* `tests/telemetry/test_perm_telemetry.py` – **Scenario:** Run a cipher with interruptors and an initial permutation. Parse the telemetry (or capture the output of `dump_telemetry`) to verify that the permutation length and hash reported

correspond to the core text length (post-interruptor) or that the telemetry explicitly labels it as full length if that's intended, ensuring consistency [55].

12. **Encoding Direction Consistency** – *File:* `tests/api/test_encoding_direction.py` – **Scenario:** Take a known plaintext and run it through `RunAPI.run` twice: once with `encoding_dir="ltr"` and once with `encoding_dir="rtl"`. For a language where direction matters (English with a reversible transliteration or Hebrew if supported), verify that the underlying indices differ as expected. If the RTL path is not implemented, the test should catch that the indices came out identical (which would be a failure unless documented). Additionally, compare the output of the API run with manually encoding the text using `Runeglish.encode_english_to_runes(..., direction=rtl)`.

13. **LM Cache Isolation** – *File:* `tests/scoring/test_lm_smoothing.py` – **Scenario:** Load a language model scorer with smoothing off, score a text to get a baseline. Then, in the same process, load another scorer on the same model file with aggressive smoothing (or different OOV handling), score something (to potentially mutate the cached tables), and then rescore the original text with the first scorer. The scores should remain the same as initially. If they differ, it indicates cache contamination. This test ensures that building a second scorer cannot side-effect the first's internal data [24].

14. **User Map3 Full-Domain Test** – *File:* `tests/ciphers/test_user_map3_domain.py` – **Scenario:** Construct a trivial `user_map3` cipher (or use the existing affine mod 29) where the effect of k1 vs k2 can be distinguished (e.g., apply key as `plaintext = (char + k1*some_constant + k2) mod A`). Choose a plaintext such that changing k1 changes the output. Run the solver or even just the cipher normalization to see if keys with k1 > 0 are ever produced. The test might directly call `cipher.encrypt/decrypt` with a sample key to ensure that part of the key is not dropped. If the solver's KeyOps only explores 0..A-1, it will never find keys with k1>0, which the test can illustrate by brute-force comparing solver output vs expectation. This test codifies the expectation that the full key space is searchable [92].

These tests cover the critical areas identified and would have caught the misbehaviors without relying on deep knowledge of the code's internals.

Open Design Questions

During the audit, certain design decisions became points of ambiguity. Before implementing fixes, these questions should be clarified:

1. **Canonical WLI Format:** Should WLI pairs always be `[pos_in_word, word_len]` everywhere (including API inputs, outputs, and internal)? The audit strongly suggests yes, to align with scoring and eliminate confusion. If any legacy reasons require `[start, end]` or `[start, length]` spans, those need to be clearly isolated or dropped.

2. **Span vs Pos Usage:** Are WLI start/end *spans* needed at all (perhaps in some output reporting or an older API)? If not, the entire span conversion logic can be removed. If yes, then conversion must be

done carefully and tests must enforce correctness. This relates to whether `(start,end)` pairs are "legacy noise" or part of a needed feature [99].

3. **Permutation semantics with interruptors:** Should the initial text permutation be defined in full-text space or core-text (post-removal) space? The docs imply core-text. If so, the API and validation should be changed to accept core-length permutations and transform or validate accordingly. Also, must we then **forbid variable interruptor counts** when a permutation is used? Probably yes, unless a more complex dynamic approach is planned.

4. **Objective direction handling:** Will the system support minimization objectives (like true NEGLOGP) in the future, or should it enforce maximize-only? The immediate fix is to ban mismatches, but longer term, perhaps solvers could have a mode or a wrapper to handle minimize (e.g., multiply scores by -1 internally). A policy decision is needed.

5. **Score dtype policy:** Is `dtype="float64"` intended to give higher precision decisions on CPU, or is it only for external interface consistency? If high precision is important, then internal casts to float32 are bugs. If not, perhaps the dtype option could even be deprecated to avoid giving false impression. Clarifying this will guide whether to refactor score storage to float64 or simply document that float32 is the enforced type.

6. **Language model cache isolation:** Should we change `_load_bin` caching to isolate by config (path + smoothing params), or simply avoid in-place smoothing? The design decision here affects performance (reloading models vs duplicating arrays) but is crucial for correctness in multi-run scenarios.

7. `user_map3` **key representation:** Define how the user (and solver) should conceptualize a two-part key. If as one number, then all parts of the system should treat it as base `A` for k2 and base `A` for k1 (like a mixed-radix). If as tuple, then allow tuple keys. This impacts the API (e.g., should `CipherConfig.key` accept a tuple for map3?), solver, and telemetry (report both components perhaps).

8. **Beam parameters source of truth:** Decide which parameter set to officially support and deprecate the other. If `expand_mode` and co. are official, remove or alias any `expand.parent_*` references in code/telemetry. Or vice versa. The current split is confusing for users and developers alike.

Addressing these questions will help ensure that fixes align with the intended design and do not introduce new inconsistencies.

Recommendations and Next Steps

In summary, to fortify RDP for hard cryptanalytic tasks, we recommend the following actions (mapping to the findings above):

- **Unify WLI Handling:** Adopt a single WLI definition (pos/len). Update `wli_from_text`, remove faulty span conversion in `coerce_wli_for_config`, and adjust tests/docs accordingly. Add runtime checks to validate WLI integrity (no pos >= len, etc.) before scoring [94].

- **Enforce WLI Alignment:** If text is permuted, apply the same permutation to WLI or disallow that combination. Implement a permutation of WLI array in `ProblemRuntime` when using `initial_text_permutation_indices` and `use_word_breaks=True` to maintain alignment [97].

- **Strengthen Interruptor Contracts:** Change `ProblemSpec` validation to accept core-length permutations if interruptors present, or explicitly forbid using permutations with interruptors unless count is fixed. Add a check when building the cipher/problem: if interruptors vary in count and a permutation is set, throw an error immediately explaining the incompatibility. Validate interruptor pools against text length on config load [57].

- **Fix Objective Direction or Ban Minimize:** Until solvers can handle minimize objectives, ensure that only maximize-type objectives are allowed. Specifically, guard against `ObjectiveFamily.NEGLOGP` usage by raising an error or automatically converting it to a maximize-friendly form (like using log-probability and flipping the solver expectation). This avoids silent direction mistakes.

- **Honor Score Dtype (Precision):** Audit all places where `np.float32` is forced. For a quick improvement, the unified scorer could return the backend's dtype (especially if a Torch model produces float64). Longer term, consider carrying double precision through critical comparisons (like plateau detection). If performance is a concern, perhaps make dtype a global toggle (with default float32) but allow advanced users to opt into float64 end-to-end.

- **WLI Missing-Data Handling:** When `use_word_breaks=True`, require valid WLI data. If `wli=None` is given (or text input with no WLI derived), either automatically disable WLI models or throw an error. This fails fast on misconfiguration and prevents the silent score flattening issue.

- **Deterministic Defaults Consistency:** Change `core/solver_engine.py` to also default to seed 0 (or better, have `RunAPI.run` always pass a seed even to legacy paths). Document this clearly so users know all runs are deterministic unless a seed is specified. Provide an option for true randomness if needed, but not as an unmarked default.

- **Seed Validation:** Remove the silent random replacement of bad seeds. Instead, if `keyops.normalize(seed)` fails, propagate an exception explaining why (length mismatch, invalid characters, etc.). Optionally, provide a utility to pad or correct seeds if that's a feature, but it must be explicit (and logged/telemetried clearly, not just debug log). Telemetry should have a field for `seed_replacements` or similar if any occur, at the very least [36].

- **Beam Solver Clarification:** Simplify the beam parameters – likely use `expand_mode` ("top" or "stochastic") and implement the stochastic parent selection if it's supposed to exist. If not, state that only "top" is supported. Align telemetry with whatever keys are actually in effect (e.g., report `expand_mode=top, top_parents=X` rather than `parent_mode`). This avoids confusion in tuning behavior.

- **Composite KeyOps Improvements:** Ensure composite key operations (like those in `composite.py`) fully respect provided key lengths and interruptor counts. Possibly split the key into sub-keys for clarity (so seeds can be two arrays: core key and interruptor pattern). If seeds are

provided as one array, at least validate the length exactly (and error clearly if not) instead of patching with random values [35].

- `user_map3` **Full Domain Support:** Modify the `GenericMapCipher` so that keys are not reduced mod A before decrypt. If keeping single-value keys, drop that mod operation and allow values up to A*A-1. Also update KeyOps hints for this cipher to use `mod = A*A`. Alternatively, refactor it to treat the key as a 2-tuple (this would be a larger change but might align with how other ciphers are handled). This will open up the full key space to the solver [85] [89].

- **Telemetry Accuracy:** Audit telemetry outputs for any other mis-reporting. We identified permutation length; another potential one is seed source (it might currently always say "provided" even if replaced). Fix these to be truthful. Telemetry is crucial for users to trust what the solver is doing.

- **Documentation and Tests:** Update documentation (especially the API usage and tutorials) to highlight any changed contracts (like WLI now being pos/len). Add the proposed integration tests (or similar) to the test suite to prevent regressions on these points. The tests listed above cover the major failure modes observed [100].

By addressing these recommendations, RDP will become more robust, **ensuring that complex multi-phase ciphers and edge-case configurations either solve correctly or fail fast with informative errors**, rather than silently misbehaving. This aligns with RDP's goal of being a "scientific instrument" for cryptanalysis – favoring transparency, correctness, and reproducibility over implicit convenience. Each fix should be validated against the new integration tests and any existing tests to maintain overall integrity.

---

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60
61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
91 92 93 94 95 96 97 98 99 100 audit_solver_blockers_2026-01-27.md

file://file_00000000c14c71f8ac2f349701e664aa